

MACHINE LEARNING



HARSHITA GOGIA

ROLL NO.: 2022PMA6908

MASTER OF SCIENCE ,MATHEMATICS

DEPARTMENT OF MATHEMATICS

NSUT, DELHI

MINOR PROJECT SUBMITTED TO

Dr. AMITA SHARMA

Harshita

Student sign.

Mentor Sign.

INDEX

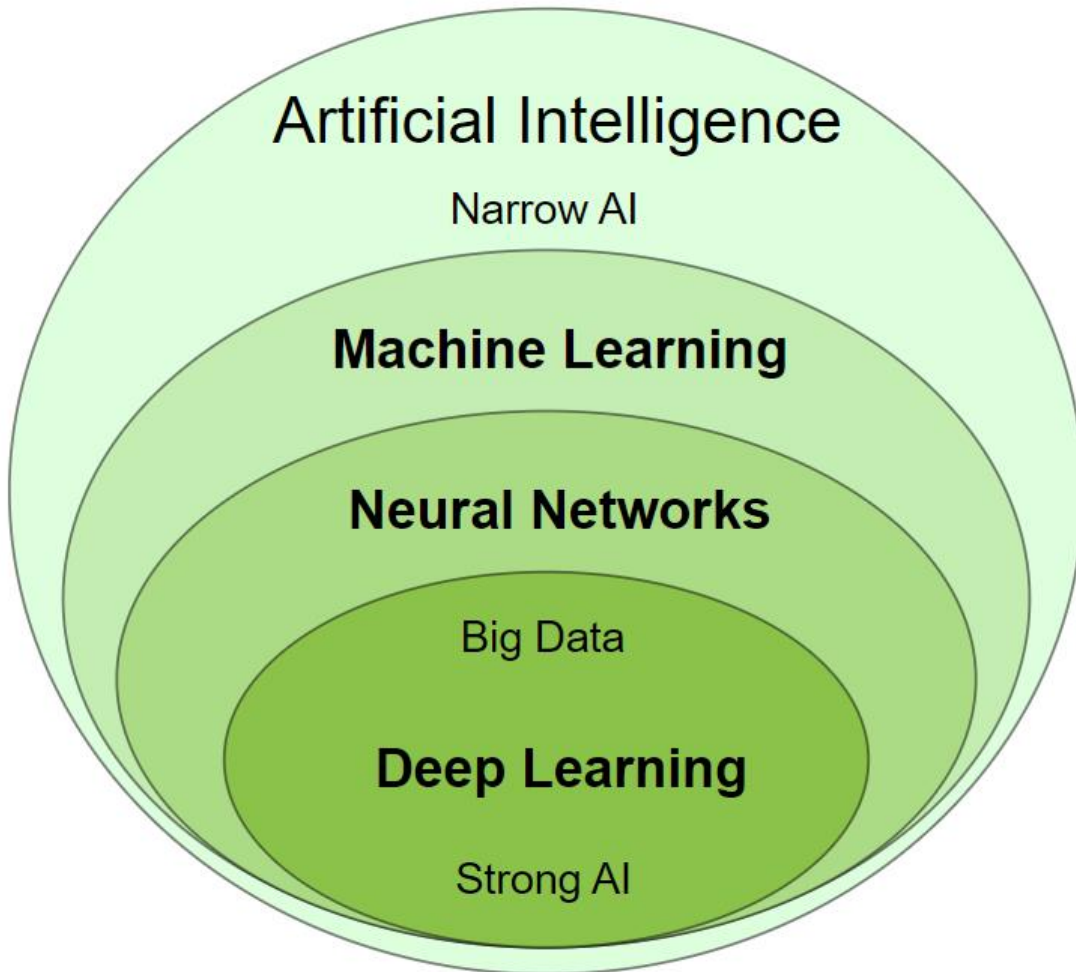
S.NO	TOPICS	PAGE NO:
1	An Introduction to Machine Learning	3-4
2	Machine Learning & Finance	5
3	Neural Network	6
4	Models on Neural Network	7
5	Types of Activation Function	8-9
6	Components of Neural Network	10
7	Working of Neural Network(Feed forward & Backward Propagation)	11-15
8	Working in Python	15-16
7	Future Direction	17
8	References	18

AN INTRODUCTION TO MACHINE LEARNING

- Machine learning is a field of artificial intelligence that empowers computers to learn and improve from experience without being explicitly programmed.
- It involves algorithms that enable systems to recognize patterns, learn from data, and make decisions or feeding predictions based on that learning.
- By large amounts of data, machine learning models can uncover insights, identify trends, and make accurate predictions or decisions in various domains, from healthcare and finance to autonomous driving and natural language processing.
- It's a transformative technology that drives innovations, automates processes, and enhances solutions across industries by enabling computers to learn and adapt without human intervention
- Machine learning's ability to handle complex and dynamic datasets makes it a powerful tool for uncovering intricate patterns and relationships.



"Learning machines to imitate human intelligence"



Traditional Programming uses known algorithms to produce data:

DATA + ALGORITHMS = RESULTS

Machine Learning creates new algorithms from data & results:

DATA + RESULTS=ALGORITHMS

MACHINE LEARNING & FINANCE!!

Algorithmic Trading: Machine learning models analyze market data to make trading decisions. They identify patterns, trends, and correlations in financial markets, enabling algorithmic trading strategies that execute transactions faster and more efficiently than human traders.

Risk Management: ML models assess and predict risks associated with loans, investments, and portfolios. They analyze historical data to identify potential risks and mitigate them, improving decision-making and reducing financial losses.

Portfolio Management: ML models optimize portfolio construction by considering various factors, including risk tolerance, diversification, and market conditions. They dynamically adjust asset allocations for better returns and reduced risk.

Insurance Underwriting: ML helps insurance companies assess risk and determine premiums by analyzing customer data, claims history, and market trends, enhancing the accuracy of underwriting processes.

NEURAL NETWORK

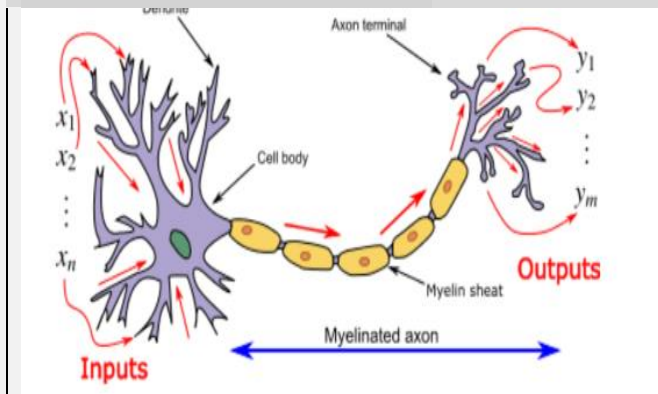
Neural Network is:

- A programming technique
- A method used in machine learning
- A software that learns from the mistakes

Neural Networks are based on human brain works .

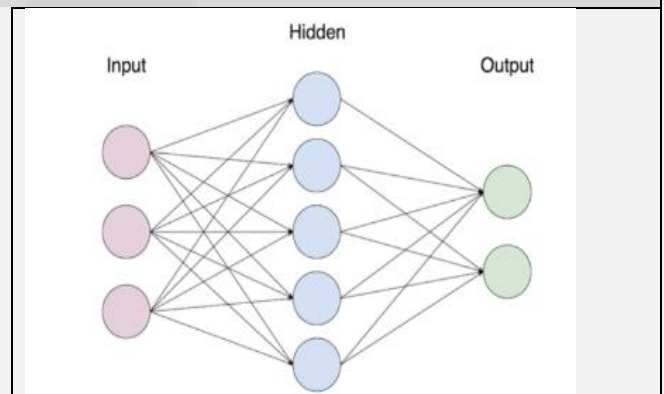
BIOLOGICAL NEURAL NETWORK

Biological Neural Network (BNN) is a structure that consists of Synapse, dendrites, cell body, and axon. In this neural network, the processing is carried out by neurons. Dendrites receive signals from other neurons, Soma sums all the incoming signals and axon transmits the signals to other cells.



ARTIFICIAL NEURAL NETWORK

Artificial Neural Network (ANN) is a type of neural network that is based on a Feed-Forward strategy. It is called this because they pass information through the nodes continuously till it reaches the output node. This is also known as the simplest type of neural network.



MODELS ON NEURAL NETWORK

1.DEEP MULTILAYER PERCEPTRONS

Deep Multilayer Perceptrons (DMLP) are neural networks consisting of multiple layers of interconnected nodes (neurons), enabling complex pattern recognition and non linear modelling. It is used in pattern recognition, function approximation etc.

2.LONG SHORT TERM MEMORY

LSTM (Long Short-Term Memory) is a type of recurrent neural network designed to handle sequence data, capable of learning long-range dependencies and retaining information over extended periods, commonly used in natural language processing, time series prediction, and speech recognition tasks.

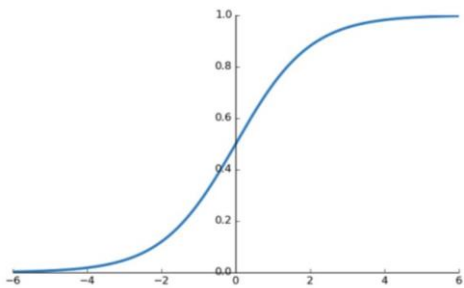
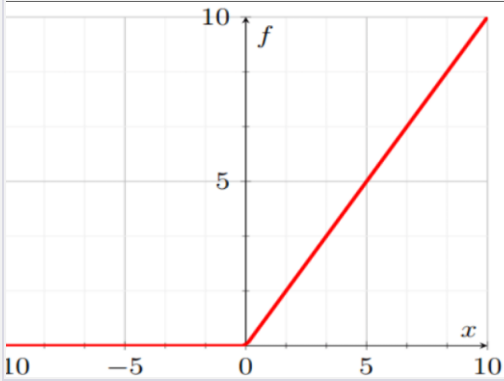
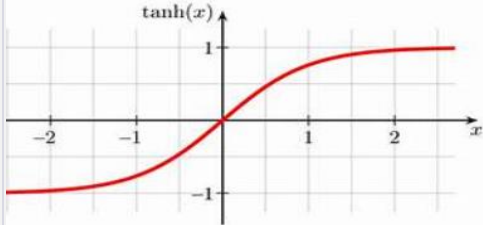
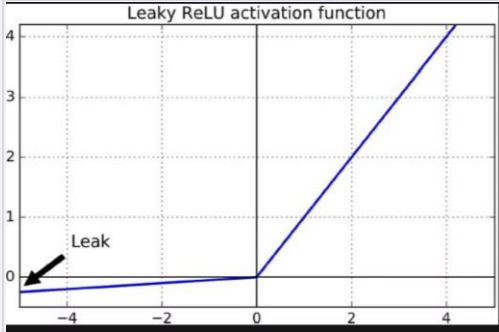
3.CONVOLUTIONAL NEURAL NETWORK

CNN (Convolutional Neural Network) is a specialized neural network adept at extracting spatial patterns from images, widely employed in image classification, object detection, and computer vision tasks due to its ability to recognize visual patterns hierarchically.

4.RANDOM FOREST

Random Forest (RF) is an ensemble learning method that aggregates multiple decision trees to make predictions, known for its robustness and versatility in classification, regression, and anomaly detection tasks across various domains, including finance, healthcare, and ecology.

TYPES OF ACTIVATION FUNCTION

Activation Function	Graph	Uses
Sigmoid $\frac{1}{1 + e^{-x}}$		Historically used in hidden layers of neural networks but less common due to vanishing gradient issues. Used in binary classification tasks where outputs need to be in the range [0, 1].
ReLU (Rectified Linear Unit) $\begin{cases} 0, & \text{if } x < 0 \\ x, & \text{if } x \geq 0 \end{cases}$		Widely used in deep learning due to simplicity and efficient training, especially in image recognition and classification tasks.
Tanh $= \frac{e^{2x} - 1}{e^{2x} + 1}$		Similar to the sigmoid but outputs values in the range [-1, 1]. Used in RNNs and LSTMs for its ability to handle varying scales of input data.
Leaky ReLU $= \begin{cases} x, & \text{if } x \geq 0 \\ \alpha x, & \text{if } x < 0 \end{cases}$		An improved version of ReLU to mitigate the "dying ReLU" problem, used where the standard ReLU might result in dead neurons.

Activation Function	Graph	Uses
Softmax $= \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$		Primarily used in the output layer of neural networks for multi-class classification problems, where it normalizes the outputs into probability distributions.

WHY WE HAVE TO USE NON-LINEAR ACTIVATION FUNCTION?

Since, we have non-linear function in real life $\therefore$ we have to use non-linear activation function. Nonlinear activation functions enable neural networks to create complex decision boundaries. For classification tasks, this flexibility in shaping decision boundaries is crucial for the network to distinguish between different classes that might not be separable by simple linear boundaries. While nonlinear activation functions are crucial for learning complex patterns, it's also worth noting that they're typically used in hidden layers. In the output layer, the choice of activation function depends on the task (e.g., Softmax for multiclass classification, sigmoid for binary classification), and sometimes linear activation functions are used, especially for regression tasks where the network is expected to predict continuous values.

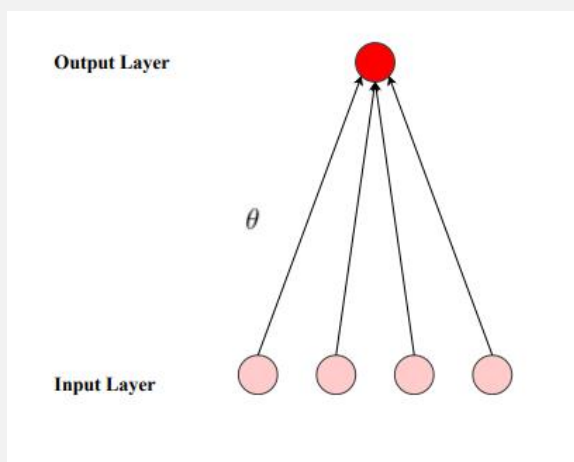
COMPONENTS OF NEURAL NETWORK

1. **Input Layer:** The first layer of the network where data is introduced. Each node in this layer represents an input feature.
2. **Hidden Layers:** Layers between the input and output layers where the computation occurs. Each neuron in these layers performs a weighted sum of inputs, applies an activation function, and passes the result to the next layer.
3. **Output Layer:** The final layer that produces the network's output based on the computations performed in the hidden layers
4. **Activation Functions:** Nonlinear functions applied to the output of each neuron, introducing nonlinearity and enabling the network to model complex relationships.
5. **Weights and Biases:** Parameters that the network adjusts during training to minimize error. They determine the strength of connections between neurons.
6. **Loss Function:** Measures the difference between predicted and actual output. It guides the network during training by quantifying the error.
7. **Optimization Algorithms:** Techniques used to update weights and biases during training, such as gradient descent, Adam, RMSProp, etc.

WORKING OF NEURAL NETWORK

1. FEEDFORWARD PROPAGATION:

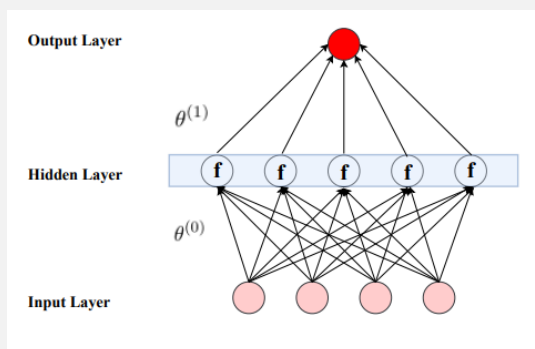
I. NEURAL NETWORK WITH NO HIDDEN LAYERS



The left fig. shows the simple possible neural network with no hidden layer. In this input layer consists of 4 neurons, viz. $\{z_1, z_2, z_3, z_4\}$. It includes intercept (θ), one w_i associated to each z_i . The

output layer aggregates the weighted signal into the forecast $\theta_0 + \sum_{k=1}^4 z_k \theta_k$; that is, the simplest neural network is a linear regression model.

II. NEURAL NETWORK WITH HIDDEN LAYERS



The model incorporates more flexible predictive associations by adding hidden layers between the inputs and output. This fig. shows 4 neuron in input

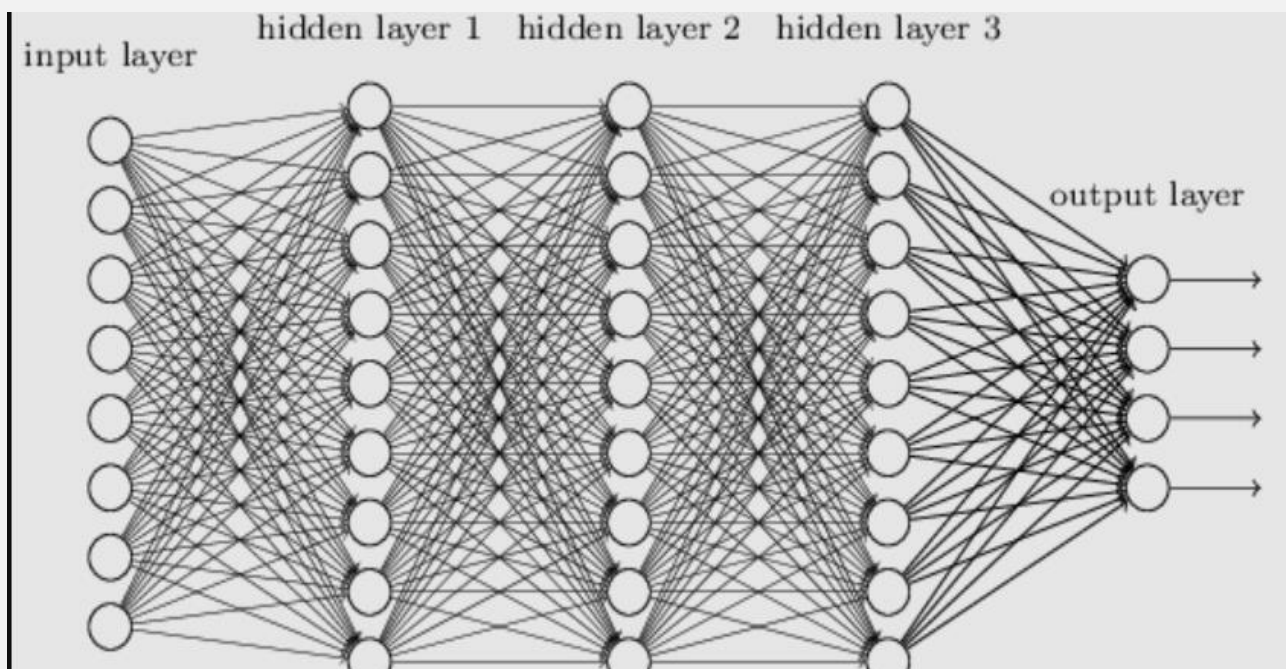
Layer, 1 hidden layer containing 5 neurons, f is a non linear activation function, 1 neuron in the output. Each neuron draws information linearly from all of the input units, just as in the simple network on the left. Then, each neuron applies a nonlinear “activation function” f to its aggregated signal before sending its output to the next layer. For example, the second neuron in the hidden layer transforms inputs into an output

$$x_2^{(1)} = f(\theta_{2,0}^{(0)} + \sum_{j=1}^4 z_j \theta_{2,j}^{(0)})$$

Lastly, the results from each neuron are linearly aggregated into an ultimate output forecast:

$$g(z; \theta) = \theta_0^{(1)} + \sum_{j=1}^5 x_j^{(1)} \theta_j^{(1)}$$

Similarly, we have *MULTILAYER NEURAL NETWORK*:



2. BACKWARD PROPAGATION:

Backpropagation is a fundamental algorithm used in training neural networks. It's a way of adjusting the weights of the network by propagating the error backward from the output layer to the input layer. Steps involved in the process are as follows:

- ✓ **Forward Pass:** During forward propagation, input data passes through the network, and calculations are performed layer by layer to generate an output. Each neuron applies a weighted sum to the input data, adds a bias term, and passes the result through an activation function to produce an output.
- ✓ **Calculating Error:** The error is calculated by comparing the network's output (predicted output) to the actual output (target output) using a chosen loss/cost function. The choice of loss function depends on the problem being solved (e.g., mean squared error for regression, cross-entropy for classification). Let's assume a simple mean squared error (MSE) loss:.

$$L = \frac{1}{2}(\hat{y} - y)^2$$

- ✓ **Backward Pass:** Backpropagation involves propagating this error backward through the network to update the weights and biases. The algorithm computes the gradient of the loss function with respect to each weight and bias using the chain rule of calculus.

Output Layer:

- Compute the gradient of the loss with respect to the output layer activations:

$$\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$$
- Compute the gradient of the loss with respect to the output layer weighted sum:

$$\frac{\partial L}{\partial (h \cdot W_2)} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial (h \cdot W_2)}$$
- If the activation function at the output layer is denoted as $\text{activation}'(\cdot)$:

$$\frac{\partial L}{\partial (h \cdot W_2)} = \frac{\partial L}{\partial \hat{y}} \cdot \text{activation}'(h \cdot W_2)$$
- Compute the gradient of the loss with respect to the output layer weights:

$$\frac{\partial L}{\partial W_2} = h^\top \cdot \frac{\partial L}{\partial (h \cdot W_2)}$$

Hidden Layer:

- Compute the gradient of the loss with respect to the hidden layer activations:

$$\frac{\partial L}{\partial (x \cdot W_1)} = \frac{\partial L}{\partial (h \cdot W_2)} \cdot \frac{\partial (h \cdot W_2)}{\partial (x \cdot W_1)}$$
- If the activation function at the hidden layer is denoted as $\text{activation}'(\cdot)$:

$$\frac{\partial L}{\partial (x \cdot W_1)} = \frac{\partial L}{\partial (h \cdot W_2)} \cdot \text{activation}'(x \cdot W_1) \cdot W_2^\top$$
- Compute the gradient of the loss with respect to the hidden layer weights:

$$\frac{\partial L}{\partial W_1} = x^\top \cdot \frac{\partial L}{\partial (x \cdot W_1)}$$

- ✓ **Update Weights:** After computing these gradients, update the weights using gradient descent or a variant (e.g., SGD, Adam) with a learning rate α :

$$W_1 \text{ new} = W_1 \text{ old} - \alpha \cdot \frac{\partial L}{\partial W_1}$$

$$W_2 \text{ new} = W_2 \text{ old} - \alpha \cdot \frac{\partial L}{\partial W_2}$$

- ✓ **Iterative Learning:** Iterative learning in neural networks involves an iterative process of feeding input data, computing outputs, comparing predictions with actual results, and adjusting network parameters (weights and biases) based on errors.

WORKING IN PYTHON: TRAINING A NEURAL NETWORK

STEP 1: IMPORT THE NECESSARY LIBRARIES

```
✓ [13] import numpy as np
```

STEP 2: TO GIVE THE INPUT VALUES

```
[18] input_value=np.array([[0,0],[0,1],[1,1],[1,0]])
      #input_value.shape
      input_value

array([[0, 0],
       [0, 1],
       [1, 1],
       [1, 0]])
```

STEP 3: TO GIVE THE OUTPUT VALUE

```
[19] output_value=np.array([0,1,1,0])
      output_value.reshape(4,1)
      output_value

array([0, 1, 1, 0])
```

STEP 4: GIVING THE INTIAL WEIGHTS

```
✓ [20] weights=np.array([[0.1],[0.2]])
      weights

array([[0.1],
       [0.2]])
```

STEP 5: GIVE THE VALUE OF BIAS

```
✓ bias=0.3
```

STEP 6: DEFINE THE ACTIVATION FUNCTION

```
def sigmoid_func(x):  
    return 1/(1+np.exp(-x))
```

STEP 7: DERIVATE THE SIGMOID FUNCTION

```
63] def der(x):  
    return sigmoid_func(x)*(1-sigmoid_func(x))
```

STEP 8: UPDATING THE WEIGHTS

```
learning_rate = 0.05 # Adjust the learning rate  
  
for epochs in range(10000):  
    input_arr = input_value  
    weighted_sum = np.dot(input_arr, weights) + bias  
    first_output = sigmoid_func(weighted_sum)  
    error = output - first_output  
    total_error = np.square(error).mean()  
  
    first_der = error  
    second_der = der(first_output)  
    derivative = first_der * second_der  
  
    t_input = input_value.T  
    final_derivative = np.dot(t_input, derivative)  
  
    weights += learning_rate * final_derivative  
    bias += learning_rate * derivative.sum()  
  
print("Final weights:")  
print(weights)  
print("Final bias:")  
print(bias)
```

```
Final weights:  
[[-0.41953547  
 [ 8.98887811]]  
Final bias:  
-4.19706343627475
```

STEP 10: PREDICTIONS

```
✓ pred=np.array([0,1])  
result=np.dot(pred,weights)+bias  
res=sigmoid_func(result)  
print(res)
```

```
[0.99177089]
```

```
pred=np.array([1,1])  
result=np.dot(pred,weights)+bias  
res=sigmoid_func(result)  
print(res)
```

```
[0.9875349]
```

FUTURE DIRECTIONS

Understanding neural networks in machine learning lays a strong foundation for applying these skills to stock prediction within the finance domain. Leveraging advanced algorithms to analyse market trends becomes pivotal in aiding investors to make informed choices. By utilizing machine learning's prowess, our aim is to foresee potential stock movements, providing valuable insights that enhance financial decision-making.

In the realm of finance, employing neural networks enables the analysis of market dynamics, offering a glimpse into future stock behavior. This application isn't just about predicting stock prices; it's about empowering individuals to allocate their funds wisely. By harnessing machine learning's predictive capabilities, investors gain a strategic edge, potentially optimizing returns and crafting more intelligent financial strategies.

The amalgamation of machine learning and finance isn't solely about predicting stock movements; it's a transformative step toward augmenting financial acumen. It offers a deeper comprehension of market intricacies, enabling stakeholders to make informed decisions based on data-driven insights. Ultimately, it seeks to revolutionize financial strategies, enabling individuals to navigate markets more astutely and optimize their investments for a more prosperous future.

REFERENCES:

- ✓ Empirical Asset Pricing via Machine Learning*by
Shihao Gu ,Bryan Kelly ,NBER Dacheng Xiu
- ✓ Portfolio optimization with return prediction using
deep learning and machine learning by Yilin Ma,
Ruizhu Han * , Weizhong Wang
- ✓ Neural Network by Edureka